

Introduction to Structured Programming KU

Session 1: Introduction to C

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

Foundations of C Programming

In this course you will learn the fundamentals:

- 💡 **Think** like a programmer
- 🧩 **Divide and Conquer** complex problems
- 🐛 **Test and Debug** your code
- ⚙️ Understand how a program **works**



Roadmap of this Session



Data Types

Operators

Conditions

Loops

Functions

I/O

Constants

Goal

By the end of this unit, you will be able to write functional C programs involving user input, logic, and output.

Data Types and Operators

Basic Data Types

C requires us to explicitly declare what type of data a variable holds.

```
int age = 25;  
float price = 19.99;  
char grade = 'A';
```

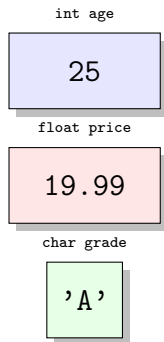
int Integers (whole numbers)

float Decimal numbers

double High-precision decimals

char Single characters

void No type (nothing)



Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum:

Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum: **13**
- Diff:

Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum: **13**
- Diff: **7**
- Prod:

Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum: **13**
- Diff: **7**
- Prod: **30**
- Quot:

Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum: **13**
- Diff: **7**
- Prod: **30**
- Quot: **3** (Truncated!)
- Mod:

Arithmetic Operators



```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;
5 int diff = a - b;
6 int prod = a * b;
7 int quot = a / b; // Integer
   Div!
8 int mod = a % b;  // Remainder
9
```

Self-Check

What are the results?

- Sum: **13**
- Diff: **7**
- Prod: **30**
- Quot: **3** (Truncated!)
- Mod: **1** ($10 = 3 \times 3 + 1$)

Output: How printf Works

printf stands for **print** formatted.

Placeholder

```
printf("Value: %d ", x );
```

Argument

69

Variable
(int x)

The Rule

The variable x "fills in" the spot marked by %d.

Common Format Specifiers

```
1 int i = 10;
2 float f = 3.14;
3 char c = 'A';
4 char s[] = "Hi";
5
6 printf("%d", i); // Integer
7 printf("%f", f); // Float
8 printf("%c", c); // Char
9 printf("%s", s); // String
10
```

Symbol	Type
%d or %i	int (Integer)
%f	float / double
%c	char (Single)
%s	String (Text)

Controlling Precision and Width

Code:

```
double p = 3.14159;

// 1. Standard
printf("%f", p);

// 2. Rounding
printf("%.2f", p);

// 3. Padding (Spaces)
printf("%7.2f", p);

// 4. Padding (Zeros)
printf("%07.2f", p);
```

Output:

3.141590 (Default)

3.14 (Rounded)

 3.14 (Width 7)

0003.14 (Zeros)

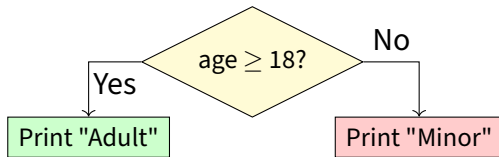
Key Takeaway:

- .2 means 2 decimal places.
- 7 means **total** characters (including dot).

Conditions

Logic Flow

```
1 if (age >= 18)
2 {
3     printf("Adult");
4 }
5 else
6 {
7     printf("Minor");
8 }
9
```



Relational Operators:

- ==, != (Equal, Not Equal)
- >, <, >=, <=

Scope: To Brace or Not to Brace?

An if statement controls the **very next** statement.

1. Block Scope (Recommended)

```
1 if (age >= 18)
2 {
3     printf("Adult\n");
4     printf("Welcome!\n");
5 }
6
```

✅ **Safe:** The curly braces {} group **both** lines together. Both run only if true.

2. Single Line (No Braces)

```
1 if (age >= 18)
2     printf("Adult\n");
3     printf("Welcome!\n");
4
```

⚠️ **Danger:** Only the **first** line is inside the if. "Welcome!" prints **always**, regardless of age!

Best Practice

Always use curly braces {}. It prevents bugs even if you only have one line.

The "Indentation Lie"

The C compiler does **not** care about your indentation. It only cares about syntax.

What you write:

```
1 int x = 0;
2 if (x > 5)
3     printf("A");
4     printf("B");
5     // Indentation suggests
6     // B belongs to if?
7
```

What the Compiler sees:

```
1 int x = 0;
2 if (x > 5)
3 {
4     printf("A");
5 }
6
7 printf("B"); // ALWAYS runs!
8
```

⚠ Remember: C is not Python! Whitespace means nothing.

The Special Case: Floating Point

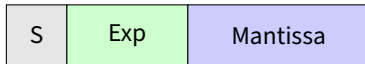


```
1 float f = 69.420f;  
2 printf("%f", f);  
3
```

Imprecision

Floats are stored in scientific notation (IEEE 754). They are **not** exact!

Rule of Thumb: Never use == with floats.
Use <, >, <=, or >=.



Computer Memory Representation
(Approximation)

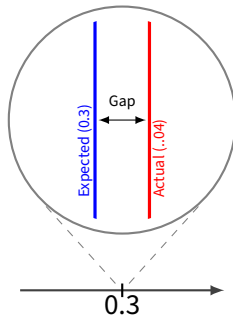
Why == Fails with Floats

Let's check: $0.1 + 0.2 \stackrel{?}{=} 0.3$

```
1 double a = 0.1;
2 double b = 0.2;
3
4 if (a + b == 0.3) {
5     printf("Math works!");
6 } else {
7     printf("Wait, what?\n");
8     // Print with high precision
9     printf("%.17f\n", a+b);
10 }
11
```

Output

```
Wait, what?
0.300000000000000004
```



The Switch-Case

Better than many `else if` statements when checking one variable.

```
1 switch (grade) {  
2     case 'A':  
3         printf("Excellent!");  
4         break;  
5     case 'B':  
6         printf("Good job!");  
7         break;  
8     default:  
9         printf("Keep trying!");  
10 }  
11
```

 **Don't forget the `break`!** Without it, code "falls through" to the next case.

Switch Case: The "Fall-Through" Trap

What happens if you remove the break? The program **does not stop!**

```
1 int level = 1;
2
3 switch (level) {
4     case 1:
5         printf("Easy ");
6         // No break here!
7     case 2:
8         printf("Medium ");
9         break;
10    case 3:
11        printf("Hard ");
12        break;
13 }
```

Output:

Easy Medium

Case 1

Falls through

Case 2

Switch Case: Rules & Limitations

switch is faster and cleaner than many ifs, but it is **strict**.

✓ Allowed

Only **Integer** types:

- int, long, short
- char (because chars are numbers!)
- enum

```
1 switch(my_char)
2 {
3     case 'A': ... // OK
4 }
5
```

✗ Not Allowed

- **Floats** (float, double)
- **Strings** ("Hello")
- **Ranges** (e.g., case > 10:)

```
1 switch(grade)
2 {
3     case > 90: ... // ERROR!
4     // Use if-else for this
5 }
6
```

The Ternary Operator

A shorthand for simple if-else.

`(condition) ? true_val : false_val;`

Example

```
// Standard
if (a > b) max = a; else max = b;

// Ternary
int max = (a > b) ? a : b;
```

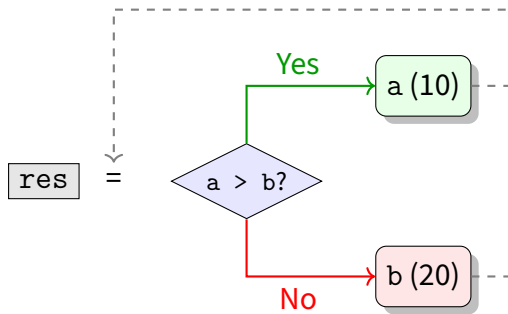
Ternary Operator: How it Flows

Unlike an if statement, the ternary operator **returns a value**.

```
int a = 10, b = 20;  
  
// The Result  
int res = (a > b) ? a : b;
```

It reads as:

"Is $a > b$? If yes, yield a . If no, yield b ."



Ternary Operator: When (Not) to Use It

👍 Good Usage

Simple assignments or return statements.

```
1 // Clean and readable
2 int min = (x < y) ? x : y;
3
4 printf("Status: %s",
5       is_online ? "On" : "Off");
6
```

👎 Bad Usage

Complex logic or **Nested** ternaries.

```
1 // The "Unreadable Mess"
2 int x = (a > b) ?
3         ((a > c) ? a : c) :
4         ((b > c) ? b : c);
5
```

(Just use if-else here!)

💡 **Rule of Thumb:** If you have to think twice to read it, don't write it.

Loops

Loop Types

We can repeat logic in 3 ways. All print 0, 1, 2, 3, 4.

1. For Loop

```
1 for (int i = 0; i < 5; i++)
2 {
3     printf("%d", i);
4 }
5
```

Best for known iteration counts.

2. While Loop

```
1 int i = 0;
2 while (i < 5)
3 {
4     printf("%d", i);
5     i++;
6 }
7
```

Best for unknown iterations.

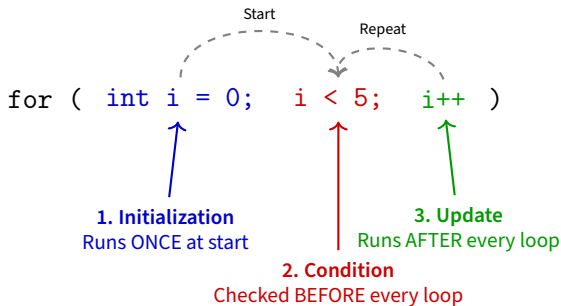
3. Do-While

```
1 int i = 0;
2 do
3 {
4     printf("%d", i);
5     i++;
6 } while (i < 5);
7
```

Always runs at least once.

Deep Dive: The For Loop

The for loop packs three steps into one line.



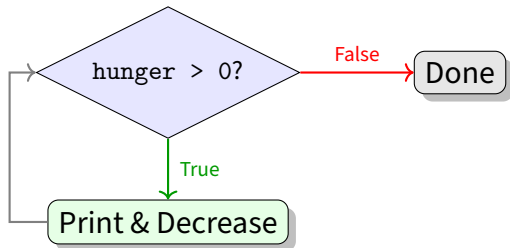
Execution Order

1. Init → **2. Check Condition (True?)** → **Body** → **3. Update** → Back to 2.

Deep Dive: The While Loop

Think of a while loop as a **Repeating If Statement**.
Code:

```
1 int hunger = 3;
2
3 while (hunger > 0)
4 {
5     printf("Eat pizza!\n");
6     hunger--;
7     // Change the variable,
8     // or infinite loop!
9 }
10
```



Side-by-Side: For vs. While

Both loops do exactly the same thing. The `while` loop just "unpacks" the logic.

1. The Compact Way (For)

```
1 // 1. Init ;
2 // 2. Check ;
3 // 3. Step
4
5 for (int i = 0; i < 5; i++)
6 {
7     printf("%d", i);
8 }
9
```

"All control logic in one line."

Same
----->

2. The Explicit Way (While)

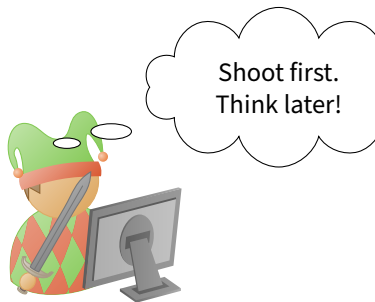
```
1 int i = 0;           // 1. Init
2
3 while (i < 5)         // 2. Check
4 {
5     printf("%d", i);
6     i++;              // 3. Step
7 }
8
9
```

"Logic distributed."

The Do-While Difference

```
1 int a = 69;  
2 do  
3 {  
4     printf("I print once!");  
5 } while(a != 69);  
6
```

Even though the condition ($a \neq 69$) is **false** immediately, the code block executes once before checking.



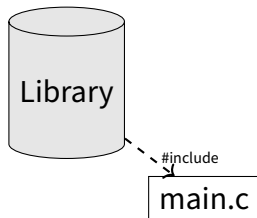
Do-While Loop being silly

Functions & Libraries

What are Includes?

C is minimal. It doesn't know how to print or do math by default. We must equip tools.

- `<stdio.h>`: Input/Output (`printf`, `scanf`)
- `<math.h>`: Math (`sqrt`, `pow`)
- `<stdlib.h>`: Utilities (Will be important for later assignments)
- `<ctype.h>`: Helpful functions for working with strings etc.

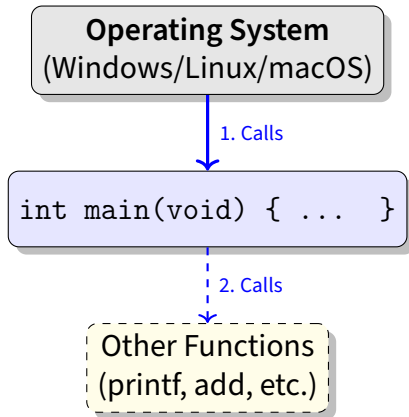


The Main Function: The Front Door

Every C program **must** have exactly one function named `main`.

```
1 #include <stdio.h>
2
3 // The "Entry Point"
4 int main(void) // We use (void) !!
5 {
6     printf("I start here!\n");
7     return 0; // The End
8 }
9
```

- Execution **always** starts at the top of 'main'.
- When 'main' finishes, the program dies.



Why return 0?

The `main` function returns an `int` (integer) back to the Operating System.

The Universal Signal

This integer is a "Mission Report":

- 0:  **Success.**
Finished normally.
- Non-Zero:  **Error.**
Something went wrong.

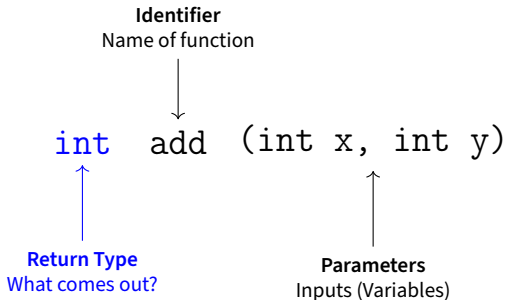
```
user@linux: $ ./program
I start here!
user@linux: $ echo $?
0
```

Terminal Check

Exit Code

Functions: The Anatomy

A function definition consists of a header and a body.



```
1 // Example
2 int add(int x, int y) {
3     return x + y; // "return" sends data back
4 }
```

Void Functions



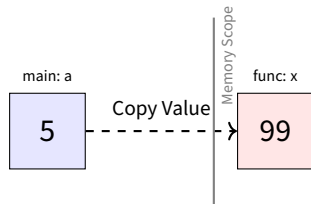
Pitfall: Void

If a function returns nothing, use void. `void print_hello() { ... }`

Pass-by-Value (Copies)

When you call a function, C passes **copies** of the values, not the original variables!

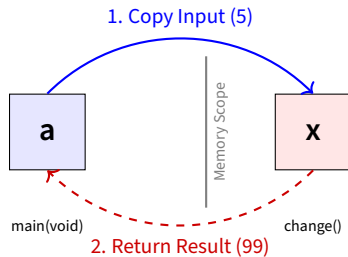
```
1 void change(int x)
2 {
3     x = 99; // Only changes
4     local copy
5 }
6 int main(void)
7 {
8     int a = 5;
9     change(a);
10    printf("%d", a);
11    // Still 5!
12 }
```



Pass-by-Value: The Fix

Since functions work on **copies**, we must **return** the new value and re-assign it.

```
1 // 1. Change return type to int
2 int change()
3 {
4     return 99; // 2. Send back
5 }
6
7 int main(void)
8 {
9     int a = 5;
10    a = change(); // 3. Catch
11    printf("%d", a); // Now 99!
12    return 0;
13 }
```



The Order Matters!

The compiler reads files from **Top to Bottom**. It cannot use what it hasn't seen yet.

The Code (main.c)

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     // Error! Who is "add"?
6     // I haven't seen it yet!
7     int res = add(5, 3);
8     return 0;
9 }
10
11 int add(int x, int y)
12 {
13     return x + y;
14 }
15
```

The Compiler Output

```
warning: implicit declaration of function
'add'
```



Common Mistakes: Implicit Declaration

If you forget the forward declaration:

Compiler Warning/Error

```
warning: implicit declaration of function 'add'
```

If you compile with flags (which you should):

```
gcc -Wall -Werror -std=c99 main.c -o program
```

...the warning becomes an error, and your code won't compile!

The Fix: Forward Declaration

We promise the compiler: "This function exists, trust me, the code comes later."

```
1 #include <stdio.h>
2
3 // 1. DECLARATION (The Promise)
4 int add(int x, int y); // Semicolon is important!
5
6 int main(void)
7 {
8     int res = add(5, 3); // 2. CALL (The Usage)
9     return 0;
10 }
11
12 int add(int x, int y) // 3. DEFINITION (The Implementation)
13 {
14     return x + y;
15 }
```

Input/Output (I/O)

Interacting with the User

```
1 int age;  
2 printf("Enter age: "); // Output  
3 scanf("%d", &age);      // Input (Note the &)  
4 printf("You are %d", age);
```

Type	Specifier	Example
Integer	%d	42
Float	%f	3.14
Character	%c	'A'
String	%s	"Hello"

Input: The Address Operator (&)

Crucial Rule: `scanf` needs to know *where* to put the data in memory.

```
1 int age;  
2  
3 // WRONG!  
4 // scanf("%d", age);  
5  
6 // CORRECT  
7 scanf("%d", &age);  
8 // "Store integer at the ADDRESS  
9   of age"
```

int age



Address: 0x1004

Put 25 Here

&

Single Character I/O

For reading/writing one character at a time, we have specialized functions.

1. getchar()

```
1 char c;  
2 printf("Press a key: ");  
3  
4 // Reads one char from stdin  
5 c = getchar();  
6
```

Often used in loops to read text!

2. putchar()

```
1 char c = 'A';  
2  
3 // Prints one char to stdout  
4 putchar(c);  
5
```

Equivalent to `printf("%c", c);`

Reading Arbitrary Input: The Pattern

To read an unknown amount of text, we loop until we hit **EOF** (End of File).
The Idiom:

```
1 int c;  
2 // Must be int to hold EOF!  
3  
4 // 1. Assign c = getchar()  
5 // 2. Compare result != EOF  
6 while ((c = getchar()) != EOF)  
7 {  
8     putchar(c); // Process char  
9 }  
10
```

Syntax Trap ⚠

!= binds tighter than =

✗ `c = getchar() != EOF`
(Assigns 1/0 to c)

✓ `(c = getchar()) != EOF`
(Assigns char to c)

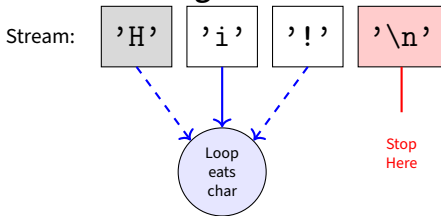
Example: Reading a Line

`getchar()` reads one character at a time from the input stream.

Goal: Count characters until Enter.

```
1 printf("Type text: ");
2 int c;
3 int count = 0;
4 // Keep reading until Newline
5 while ((c = getchar()) != '\n'
6        && c != EOF)
7 {
8     count++;
9 }
10 printf("Length: %d", count);
11
```

Visualizing the Stream:



💡 **Note:** This is how you "clear" the buffer if `scanf` left junk behind!

The Problem with scanf

scanf("%d", &val) is great for simple numbers, but fragile for user input.

Scenario: User types "twelve" instead of 12.

```
1 while(1)
2 {
3     printf("Number: ");
4     if (scanf("%d", &val) == 1)
5     {
6         break; // Success
7     }
8     // ERROR! Buffer not cleared!
9 }
```

The "Sticky" Buffer

scanf does **not** consume input if it doesn't match the format specifier. It leaves "twelve" for the next call, causing an infinite loop.

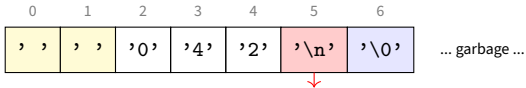
Better Approach: Read the *entire line* as a string first, then try to make sense of it.

Safe Input: fgets

fgets reads a string until it hits a Newline (\n) or the buffer is full.

```
1 char buffer[1024]; // Make it big enough (simplified)
2 printf("Input: ");
3 // 1. Where to store? (buffer)
4 // 2. How much max? (1024)
5 // 3. From where? (stdin)
6 fgets(buffer, 1024, stdin);
7
```

Visualizing the Buffer after inputting "042\n":



Be careful! fgets keeps the Enter key!

Processing the String: Whitespace & Zeros

Once input is in a string buffer, we can analyze it safely without crashing the program.

1. Skipping Whitespace

Computers read left-to-right. We can use a simple loop to find the start.

```
1 int i = 0;
2 // Requires <ctype.h>
3 while (isspace(buffer[i]))
4 {
5     i++;
6 }
7 // buffer[i] is now the
8 // first real char
9
```

2. Parsing Numbers

How to turn string "42" into int 42?

- `atoi(buffer)`: Simple, but no error checking (0 could mean error or zero).
- `strtol(...)`: **The Pro Way**. Handles leading whitespace, +/- signs, and reports errors.

Processing the String: Whitespace & Zeros



Hint: Leading Zeros

Standard functions like `strtol` automatically handle leading zeros (e.g., "007" becomes 7). You usually don't need to remove them manually!

Edge Case: Input Too Long?



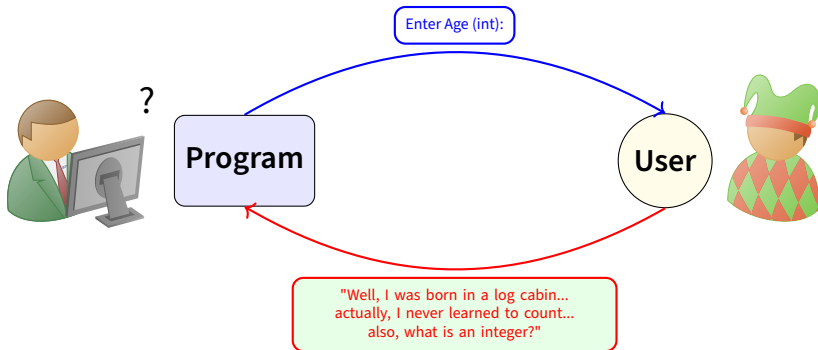
What if we ask for 10 chars, but the user types a novel?

The Problem: `fgets` stops reading when the buffer is full, leaving the rest of the line in the stream.

The *next* `fgets` call will read the leftovers immediately (no wait), causing an infinite loop or double printing.

```
1 char buf[10];
2 fgets(buf, 10, stdin);
3
4 // Input: "Hello World!"
5 // Buffer: "Hello Wor\0"
6 // Stream: "ld!\n" <- Waiting!
7
```

Input Validation: Never Trust the User Inputs



The Lesson

`scanf` expects a number. If it gets a nonsense input, it crashes or loops infinitely.
Always check return values!

Edge Case: Input Too Long?

The Fix: Detect & Flush. The `strchr()` function finds the first occurrence of a character in a string.

```
1 // 1. Check if Newline is missing (We are in a Loop here)
2 if (strchr(buffer, '\n') == NULL)
3 {
4     // 2. Consume rest of stream manually
5     int c;
6     while ((c = getchar()) != '\n' && c != EOF);
7
8     printf("Invalid input!\n");
9     continue; // Restart loop
10 }
11
```

Strict Parsing: atoi vs strtol

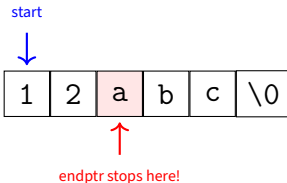
Why `atoi("12abc")` returns 12 (Valid), but we want "Invalid Input".

The Weak Way: `atoi`

- Stops at first non-digit.
- Returns 0 on error (ambiguous).
- Ignores trailing garbage.

The Robust Way: `strtol`

```
1 char *endptr;  
2 long val = strtol(buf, &endptr, 10);  
3 // Read the docs on what arguments strtol  
   takes !!  
4
```



Summary: The "Input Hell" Checklist

To survive the automated tests (and real users), your input loop needs 5 checks:

- 1 Prompt & Flush:** `printf("> "); fflush(stdout);` (Vital for tests!)
- 2 Read Safely:** Use `fgets`. Handle `NULL` (EOF).
- 3 Check Overflow:** Did we get the whole line? If not, flush `stdin`.
- 4 Trim & Empty Check:** Remove whitespace. If string is empty (User just hit Enter) → Invalid.
- 5 Strict Parse:** Use `strtol`.
 - Is `val` in range (0 - 5M)?
 - Is `*endptr` correct?

Only if ALL pass, break the loop. Otherwise: Print Error & Continue.

Constants

Constants: The `const` Keyword

The `const` keyword tells the compiler: "This variable's value cannot be changed."

```
1 int main(void)
2 {
3     // A standard variable
4     int score = 10;
5     score = 20; // OK
6
7     // A Constant
8     const double PI = 3.14159;
9
10    // COMPILER ERROR!
11    PI = 3.0;
12 }
```

PI

3.14159



Read Only

Constants: The `const` Keyword

ONLY Constants are allowed to be global (in this course)

```
1 #include <stdio.h>
2 const double PI = 3.14159;
3
4 int main(void)
5 {
6     ...
7 }
8
```

PI

3.14159



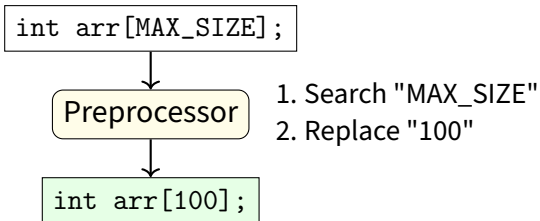
Read Only

Constants: The Preprocessor (#define)

#define is NOT a variable. It is a **Search and Replace** command that runs *before* compilation.

Your Code:

```
1 #define MAX_SIZE 100
2
3 int arr[MAX_SIZE];
4 // MAX_SIZE does not
5 // exist in memory!
6
```



Comparison: `const` vs `#define`

`#define` (Macro)

- **No Type Safety** (Just text).
- No memory address.
- Good for: Array sizes, magic numbers.

```
1 #define BUFFER_SIZE 1024
2 char buf[BUFFER_SIZE];
3
```

`const` (Variable)

- **Type Safe** (int, float, etc).
- Has a memory address.
- Good for: Physics constants, fixed values.

```
1 const float gravity = 9.81f;
2
```

Conclusion and Hints

Don't Panic!



Programming is hard at first.

- Start simple (Hello World, Calculator, Online Examples, ...)
- **Only start with the assignments after having checked and understood the learning goals!**
- Break problems down
- **Google is your friend** (but don't copy-paste blindly)
- Practice makes perfect!

Don't Cheat!



Programming is hard at first.

- But it is **not** an excuse to cheat!
- Do not steal code from online sources or colleagues!
- **We treat the involvement/help of LLMs as a natural person!**

Don't Cheat!



Programming is hard at first.

- Every usage of LLMs has to be properly cited (Prompt + Link to the shared chat)
- Failing to do so will result in a **cheating attempt!**
- Last semesters many students were caught!
- Up to 20% of your code is allowed to be cited!

Questions?

Introduction to Structured Programming KU

Session 1: Introduction to C

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security